实验报告

> 王峥 2211267
>
> 李佳璐2211985
>
> 黄煜斐 2213028

《Practical Deep Dispersed Watermarking with Synchronization and Fusion》

由字节跳动产品安全多媒体与AI安全团队发表于**ACM MM 2023**，提出了一种**基于深度学习的图像离散水印方案(DWSF)**，该方案能够有效提高图像水印在实际应用中的鲁棒性和隐蔽性，可促进深度图像水印技术在数字版权、内容可信和数据安全等防护体系建设中发挥作用。

论文代码仓库：https://github.com/bytedance/DWSF

论文阅读分析

问题描述

1.1 定义

盲水印技术旨在将特定标识信息以不可见的方式嵌入多媒体内容（如图像、视频）中，广泛应用于版权保护、溯源追踪等场景。随着人工智能生成内容（AIGC）的快速发展，盲水印还可用于标识 AIGC 内容，防止滥用。当前深度学习驱动的盲水印方法虽取得一定进展，但在实际应用中仍存在两大核心挑战：

1. **高分辨率图像适应性不足**：多数现有方法仅针对固定低分辨率图像（如 128×128）设计，而现实场景中图像分辨率普遍较高（如 OpenImages 数据集中多数图像分辨率超过 2000×2000），导致模型在高分辨率场景下性能下降。
2. **几何攻击鲁棒性缺失**：传统方法主要验证非几何攻击（如 JPEG 压缩）的鲁棒性，忽略了常见几何攻击（如旋转、缩放、裁剪）及组合攻击（如 Rotate&JPEG）的影响。几何攻击会导致水印同步失效，严重影响解码准确性。

1.2 现有技术分类

- 传统水印方法：基于空间域或变换域（如 DCT、DWT）手工设计特征嵌入水印，对复杂攻击鲁棒性有限。
- 深度学习方法：如 HiDDeN、TSDL、MBRS 等，通过端到端网络自适应嵌入水印，但存在分辨率限制和几何攻击短板。

1.3 评价指标

- 视觉质量：峰值信噪比（PSNR）、结构相似性（SSIM），衡量水印嵌入后图像的失真程度。
- 文件大小增量：水印嵌入导致的文件字节增长率（Byte Increase Rate），反映存储与传输效率。
- 鲁棒性：位准确率（Bit Accuracy），衡量攻击后水印解码的正确性；位校验准确率（Bit Check Accuracy），结合 CRC 校验评估实际可用性。

1.4 发展历史

盲水印技术从早期基于 LSB（最低有效位）的空间域方法，发展到变换域（如 DCT、DWT）嵌入，再到近年来深度学习驱动的端到端方案（如 HiDDeN 提出的自编码器架构）。然而，现有深度学习方法仍局限于固定分辨率和非几何攻击场景，未能有效解决高分辨率和几何攻击下的实用性问题。

论文摘要

文章作者提出一种实用的深度分散式水印框架 DWSF（Dispersed Watermarking with Synchronization and Fusion），旨在解决现有方法在任意分辨率图像和复杂攻击下的局限性。核心创新包括：

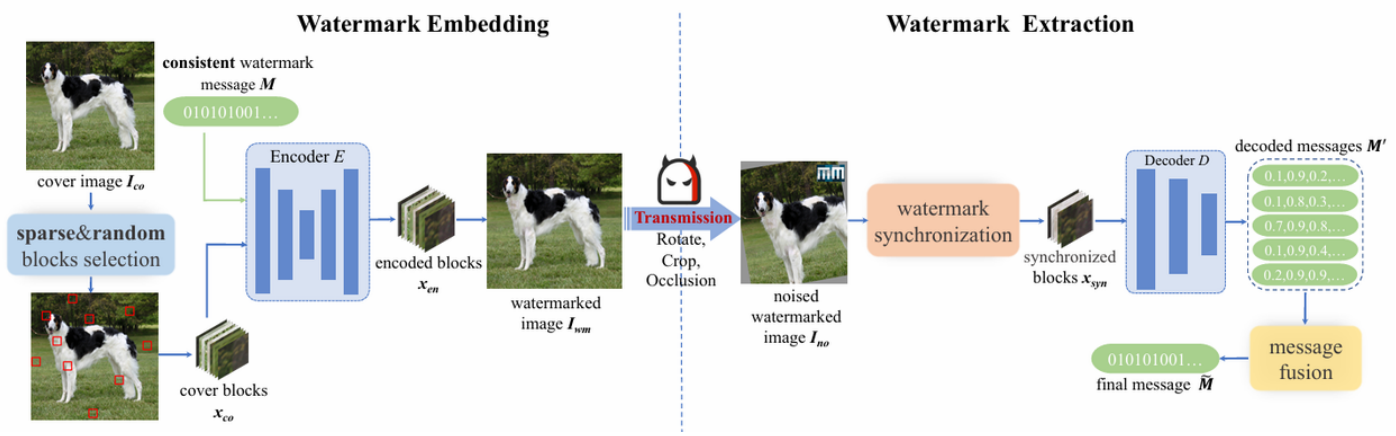
1. 分散嵌入方案：从覆盖图像中稀疏随机选择多个固定尺寸小块，嵌入一致水印信息，减少对图像的整体修改，提升视觉质量并自然抵抗局部擦除攻击（如裁剪、遮挡）。
2. 水印同步模块：通过分割模型定位编码块，结合几何变换参数估计与校正，解决旋转、缩放等几何攻击导致的同步问题。
3. 消息融合策略：基于解码消息的相似性聚合结果，过滤异常值，提升复杂攻击下的解码可靠性。

实验表明，DWSF 在 ImageNet、LabelMe、OpenImages 数据集上，面对 11 种单攻击和 6 种组合攻击，平均位准确率较先进方法提升 5.28%（单攻击）和 5.93%（组合攻击），同时文件大小增量更小、视觉质量更优。

核心内容

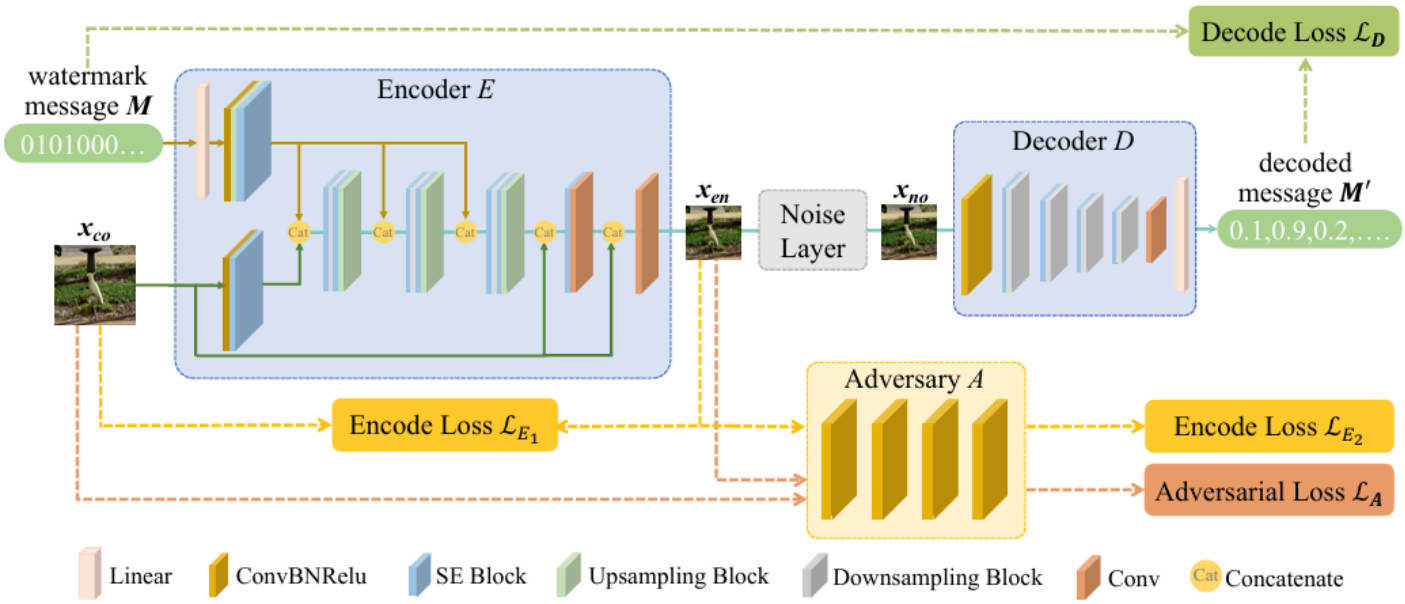
3.1 DWSF 框架概述

DWSF 整体流程如图 1 所示，包含嵌入、传输（攻击）、提取三个阶段：



1. 嵌入阶段：随机选择覆盖图像中的多个小块，通过编码器嵌入一致水印消息，生成水印图像。
2. 传输阶段：水印图像可能遭受几何攻击（如 Rotate、Crop）和非几何攻击（如 JPEG、Gaussian Noise）。
3. 提取阶段：同步模块定位并校正编码块，解码器提取消息后通过融合策略生成最终结果。

模型训练流程



为实现水印嵌入的高隐蔽性与提取鲁棒性，DWSF 通过端到端对抗训练优化模型。如图所示，Encoder 接收原始图像与水印消息，经多层网络嵌入水印；Noise Layer 模拟攻击生成受扰图像，驱动 Decoder 精准提取水印；Adversary 模块通过对抗损失约束 Encoder，让水印更难被察觉。多损失函数、解码损失、对抗损失协同优化，保障水印 “藏得好、提得出”。

3.2 分散嵌入（Dispersed Embedding）

3.2.1 设计动机

现有方法将水印嵌入整图，导致高分辨率图像修改量大、视觉失真明显，且易受局部擦除攻击影响。分散嵌入通过以下特性解决问题：

- 稀疏性：仅修改图像中少量固定尺寸块（面积占比 < 25%），降低像素修改范围，PSNR 较 HiDDeN 提升 12.16dB（LabelMe 数据集）。
- 随机性：嵌入位置随机选择，避免攻击者针对性擦除，安全性高于固定位置嵌入。
- 一致性：每个块嵌入相同消息，通过多块解码结果聚合提升鲁棒性。

3.2.2 实现方法

1. 块选择：从覆盖图像I_{co}中随机选取非重叠的h×w块（默认 128×128），总覆盖面积≤Q%（Q=25）。

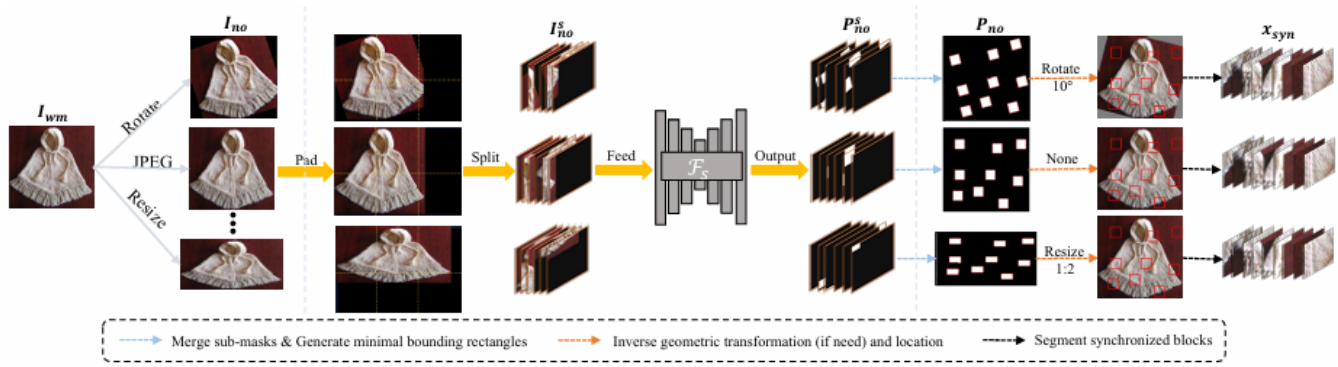
2. 编码器架构：基于 SE（Squeeze-and-Excitation）模块，将长度小于L的二进制消息嵌入块X*co*，其中α平衡像素级与结构级损失。损失函数为：

$$\mathcal{L}_{E1} = MSE(x_{co}, E(x_{co}, M)) + \alpha \cdot MSSSIM(x_{co}, E(x_{co}, M))$$

3. 噪声层：训练时模拟 5 种非几何攻击（JPEG、GN、GF、Color、Dropout）及无攻击（Identity），增强模型泛化能力。

3.3 水印同步模块（Watermark Synchronization Module）

几何攻击（如旋转、缩放）会破坏编码块的位置和形状，导致解码器无法正确提取消息。同步模块需解决两大问题。**块定位**：在攻击后图像中识别编码块区域。**几何校正**：估计并逆变换几何参数（如旋转角度、缩放比例），恢复编码块原始形态。



1. 分割模型：采用U^2-Net 轻量级网络，以 BCE 损失和 IoU 损失训练，预测编码块掩码。其中G^s为嵌入掩码真值

$$\mathcal{L}_S = BCE(P^s, G^s) + \gamma \cdot IoU(P^s, G^s)$$

2. 推理策略（Pad&Split）：
   - 对任意分辨率噪声图像I*no*，先填充至

     $$(H + H_s - H\%H_s) \times (W + W_s - W\%W_s)$$

     再分割为H_s×W_s，(H_s=W_s=512）子图。
   - 子图输入分割模型生成子掩码，合并后裁剪得到完整掩码P*no*。
3. 几何校正：
   - 基于P*no*的最小包围矩形估计几何参数（如旋转角度），过滤异常值后取平均。
   - 逆变换校正图像，提取同步块\(x_{syn}\)送入解码器。

3.4 消息融合策略（Message Fusion）

分散嵌入生成多个解码消息，高强度攻击下部分消息可能失真，直接平均会引入偏差。融合策略通过相似性筛选可靠消息，流程如下：

1. 差异计算：对N个解码消息Mi，计算任意两者的位差异：

$$\mathcal{D}_{i,j} = \sum(Binary(M_i') - Binary(M_j'))^2$$

2. 分组筛选：设置位差异阈值T（T=5），寻找满足

$$|S_{\bar{i}_t} \leq t| \geq K \quad (K=2)$$

的消息组，其中为差异≤t的消息集合。

3. 结果聚合：对筛选后的消息取平均并二值化，得到最终消息与简单平均（Mean）或最优选择（Min）相比，融合策略在 ImageNet 数据集上平均位准确率达 99.45%，仅比理论上限（Min）低 0.16%，有效过滤了异常解码结果。
4. 为验证消息融合策略的鲁棒性，在 ImageNet 数据集上对比 Min（最优单块解码）、Mean（简单平均）与本文 Message Fusion（Ours）的位准确率。如图所示：

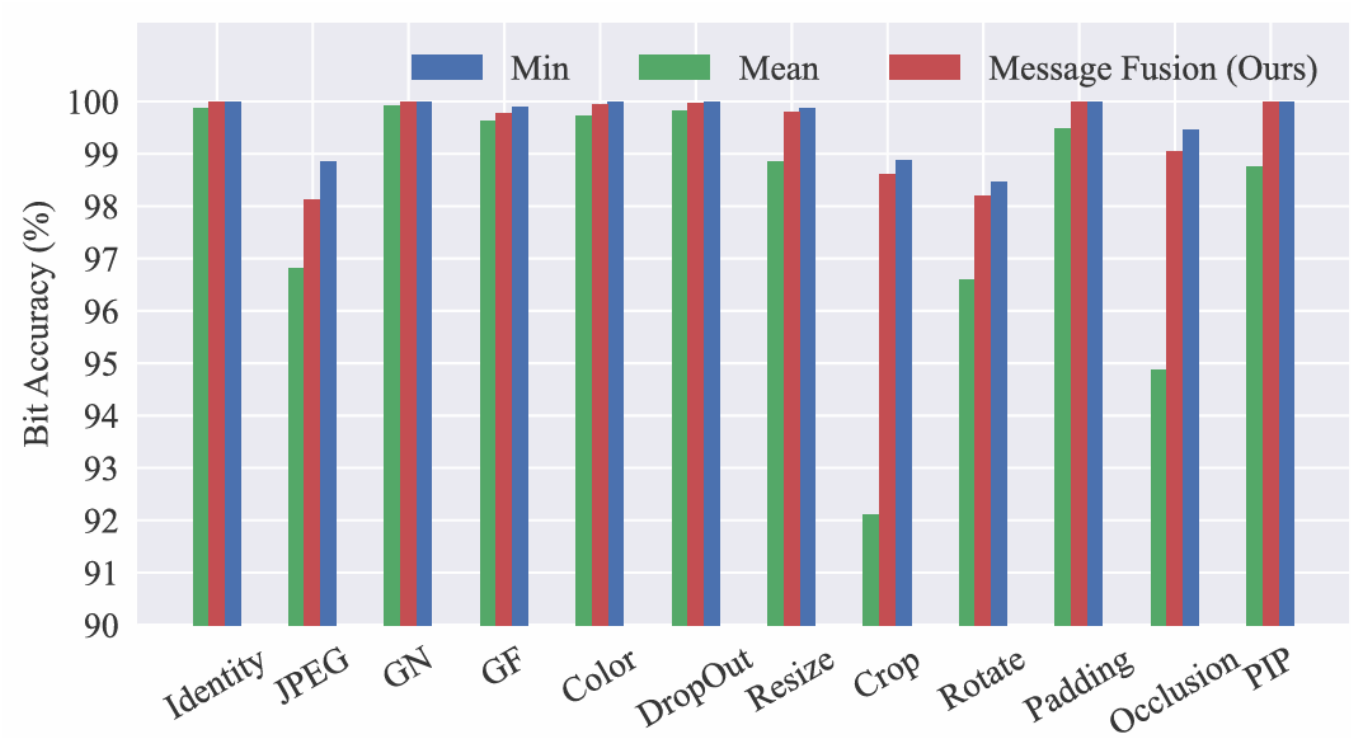


Figure 6: Bit Accuracy (%) of the Min, the Mean and our message fusion on ImageNet.

实验方法与结果

4.1 实验设置

4.1.1 数据集

1. COCO 数据集

- 数据规模：随机采样 40,000 张图像。
- 处理方式：
  - 从图像中随机裁剪 128×128 的小块，作为水印模型（编码器、解码器、对抗模块）的输入。
  - 用于训练模型嵌入与提取水印的核心能力，模拟真实场景下的图像块操作。
- 作用：作为主体训练集，让模型学习在标准尺寸图像块中嵌入水印，并抵抗非几何攻击（如 JPEG 压缩、噪声等）。

2. 水印图像的子区域

- 处理方式：裁剪 512×512 的子区域，作为分割模型（Watermark Synchronization Module）的训练数据。
- 作用：训练分割模型定位水印块，学习识别嵌入水印的区域，为几何攻击下的同步校正提供基础。

2. ImageNet、LabelMe、OpenImages

- 数据规模：各 1000 张原始分辨率图像（不缩放至固定尺寸）。
- 处理方式：
  - 直接使用原始分辨率图像，不做统一缩放（仅 MBRS 需缩放到 128×128）。
  - 用于评估模型在任意分辨率下的泛化能力，尤其是高分辨率场景（如 OpenImages 中多数图像分辨率 > 2000×2000）。
- 作用：
  - ImageNet：验证模型在自然图像中的鲁棒性。
  - LabelMe：测试模型在标注图像（含物体边界）中的表现。
  - OpenImages：重点验证高分辨率图像下的适应性，评估几何攻击（如旋转、裁剪）和组合攻击的抵抗能力。

4.1.2 对比方法

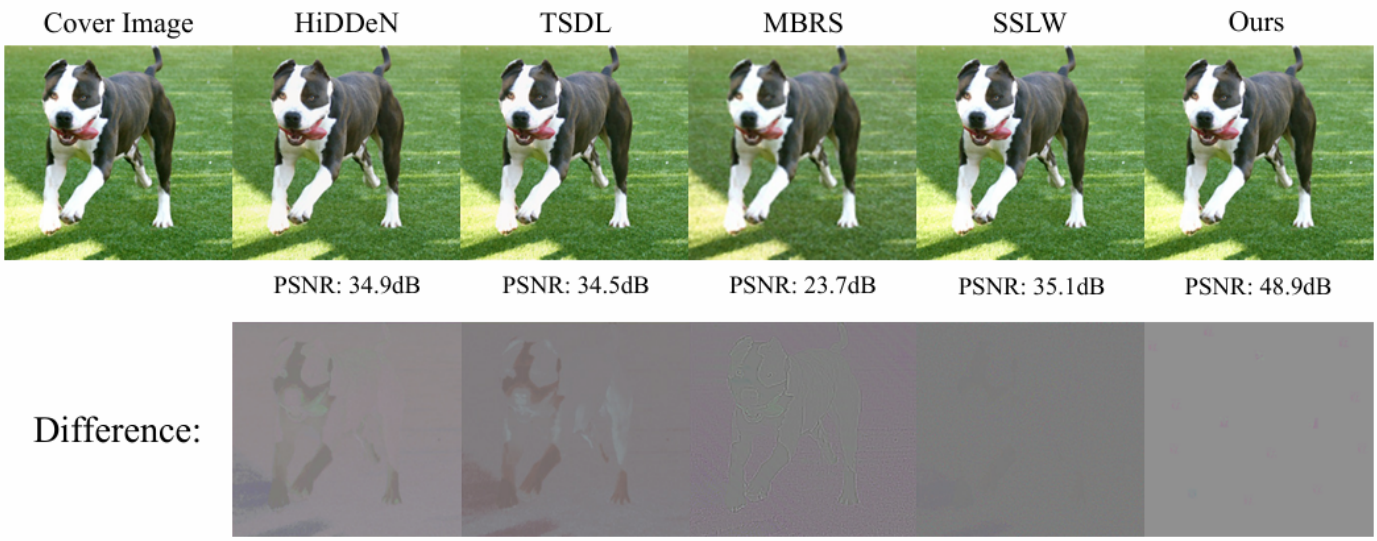
- 先进深度学习方法：HiDDeN、TSDL、MBRS、SSLW。
- 水印模型：AdamW 优化器，学习率 1e-4，训练 100 轮，批量大小 64。
- 分割模型：批量大小 24，训练 100 轮，嵌入区域 PSNR 裁剪至 35dB。

4.2 实验结果

4.2.1 视觉质量与文件大小

模型	PSNR (dB)			Byte Increase Rate (%)		
	ImageNet	LabelMe	OpenImages	ImageNet	LabelMe	OpenImage
HiDDeN	35.9	35.22	35.58	10.27	12.17	29.08
TSDL	34.8	34.76	34.94	2.74	8.3	27.49
MBRS	24.14	23.96	24.14	9.2	11.95	8.82
SSLW	35.1	35.03	35.16	19.67	22.48	45.95
DWSF (Ours)	**42.07**	**47.38**	**46.12**	**2.41**	**1.3**	**1.9**

DWSF 的 PSNR 比 HiDDeN 高 6.17-12.16dB，文件大小增量降低 80% 以上，得益于分散嵌入对图像修改的限制。同时作者也通过可视化方式向我们展现了水印嵌入的视觉差异



4.2.2 单攻击鲁棒性

数据集	模型	Identity	JPEG	GN	GF	Color
ImageNet	DWSF (Ours)	100	98.11	100	99.78	99.95
LabelMe	DWSF (Ours)	100	99.74	100	100	100
OpenImages	DWSF (Ours)	100	99.52	100	99.94	99.95

DWSF 在各类攻击下平均位准确率超 99%，尤其在几何攻击（如 Rotate）和组合攻击中优势显著，同步模块有效解决了几何失真问题。

4.2.3 组合攻击鲁棒性

攻击类型	ImageNet				LabelMe	
	HiDDeN	TSDL	MBRS	DWSF (Ours)	HiDDeN	TSDL
Color&JPEG	72.19	52.3	96.69	**96.41**	72.86	52.4
Crop&JPEG	70.76	52.15	67.65	**95.47**	72	52.62
Crop&Resize	89.57	75.27	67.64	**96.72**	93.07	79.15
AVG.	72.99	55.75	76.61	**93.69**	74.3	56.52

组合攻击下，DWSF 位准确率比次优方法 SSLW 高 5.93%，分散嵌入与消息融合的联合作用有效抵抗了多重攻击的叠加影响。

4.3 消融实验

4.3.1 同步策略对比

策略	JPEG	GN	GF	Color	Rotate	AVG.
Scale	83.61	98.23	97.54	98.12	68.97	89.29
Pad&Split	**98.11**	**100**	**99.78**	**99.95**	**98.2**	**99.45**

Pad&Split 策略在 JPEG 攻击下比 Scale 策略位准确率高 14.5%，证明分块处理能减少分辨率缩放引入的额外失真，提升分割精度。

4.3.2 消息融合策略有效性

策略	Identity	JPEG	Rotate	Crop	AVG.
Mean	100	97.86	97.47	97.99	98.33
Min	100	98.11	98.2	98.61	98.73
DWSF 融合	100	98.11	98.2	98.61	98.73

融合策略与 Min 策略性能接近，但无需预知最优消息，更具实用性，平均位准确率仅比 Min 低 0.16%。

总结

通过阅读论文，认识到作者设计DWSF 通过分散嵌入、水印同步与消息融合三大创新，首次实现了任意分辨率图像下对几何攻击和组合攻击的高效抵抗，在视觉质量、文件大小和鲁棒性三个维度均超越现有先进方法。该框架为盲水印技术在 AIGC 标识、版权保护等实际场景的应用提供了重要突破。

论文复现实验

模型复现过程

> 环境：AutoDL-RTX 3090 * 1卡
>
> 数据集：coco2017中随机选取的3200张图片

1.训练编码器 - 解码器模型

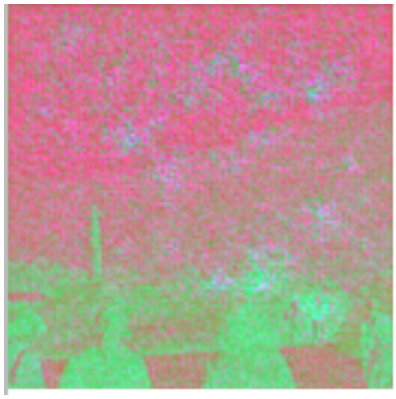
```
/root/miniconda3/lib/python3.8/site-packages/torch/nn/functional.py:3631: UserWarning: Default upsampling behavior when mode=bilinear is change
d to align_corners=False since 0.4.0. Please specify align_corners=True if the old behavior is desired. See the documentation of nn.Upsample fo
r details.
  warnings.warn(
>>>>>>>Train Epoch: 0   D_loss: = DisF:22.501304626464844 + DisT:13.521808624267578
        En_De_loss: 17.123613357543945 = Gen:14.361056327819824 + En:31.885379791259766 + De:10.732178688049316
=======evaluating============
tensor([1., 0., 0., 0., 0., 1., 0., 1., 1., 0., 1., 1., 0., 0., 0., 0., 0., 0.,
        1., 1., 1., 1., 0., 0., 1., 0., 0., 1., 1., 0.], device='cuda:0')
tensor([0.0000, 0.5080, 0.0000, 0.1727, 0.0000, 0.0000, 0.0768, 0.0918, 0.3120,
        0.0000, 0.1107, 0.0505, 0.0000, 0.0670, 0.0000, 0.0000, 0.1503, 0.1380,
        0.0000, 0.0000, 0.1748, 0.1964, 0.0844, 0.0241, 0.3007, 0.0000, 0.0213,
        0.1734, 0.0000, 0.0000], device='cuda:0')
val loss: tensor(0.6760, device='cuda:0')  gen loss: tensor(0.5805, device='cuda:0') encode loss: tensor(1.1594, device='cuda:0') decode loss:
tensor(0.4436, device='cuda:0')
loss: 0.6744171977043152
error_bit: 0.5090277777777777
error_message: 1.0
psnr: 8.77920913696289
ssim: 0.0757322907447815
save on epoch-0
Time(epoch-0):28.859779119491577
=======training============
Train Epoch: 1 [0/25]   D_loss: 1.3460516333580017 = DisF:0.8266576528549194 + DisT:0.5193939805030823
        En_De_loss: 0.6334020495414734 = Gen:0.5808637738227844 + En:1.1332066059112549=MSE-0.5010032653808594+MSSIM-126.44068145751953, De:0.4
0617984533309937
```

```
=======evaluating============
tensor([0., 1., 1., 1., 1., 1., 1., 1., 0., 1., 1., 1., 0., 0., 1., 1., 1., 0.,
        0., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., 0.], device='cuda:0')
tensor([0.4534, 0.4885, 0.6924, 0.6830, 0.8899, 0.3985, 0.5473, 0.3382, 0.1483,
        0.3014, 0.4479, 0.0987, 0.5148, 0.7009, 0.5216, 0.6482, 0.7437, 0.5190,
        0.3426, 0.5442, 0.5708, 1.0258, 0.4601, 0.0000, 0.1769, 0.2515, 0.5351,
        0.7943, 0.9749, 0.2264], device='cuda:0')
val loss: tensor(0.3675, device='cuda:0')  gen loss: tensor(0.6507, device='cuda:0') encode loss: tensor(0.8031, device='cuda:0') decode loss:
tensor(0.2062, device='cuda:0')
loss: 0.36033299565315247
error_bit: 0.30019841269841274
error_message: 1.0
psnr: 12.269367218017578
ssim: 0.06981004029512405
save on epoch-99
Time(epoch-99):28.22805690765381
```

输出**训练模型**以及**img**（观察模型在不同训练阶段对图像的编码和解码效果，以及噪声处理的影响）

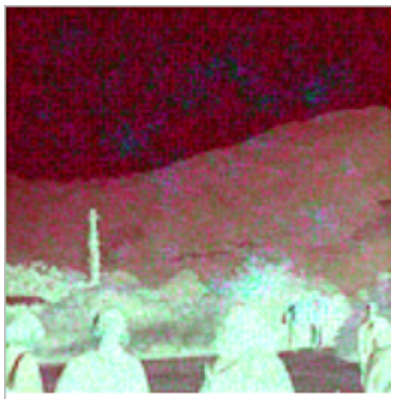
- 原始图像（`ori.png`）：输入到编码器之前的原始图像，用于和编码后的图像进行对比

- 编码后图像（`en.png`）：保存了编码后的图像。编码过程是将原始图像和随机生成的消息进行结合，生成包含消息的编码图像。



- 编码并经过噪声处理后的图像（`en_dis.png`）： 保存了经过噪声层处理后的编码图像。噪声层模拟了图像在传输或存储过程中可能遇到的各种干扰，如 JPEG 压缩、高斯噪声等。

- 噪声图像（`noise.png`）： 保存了编码图像和原始图像之间的差异图像，即编码过程中添加的噪声信息。



2.训练分割模型

1. 在训练分割模型之前，需要生成带水印的图像和掩码

```
(base) root@autodl-container-a3c347aab8-8e755a96:~/autodl-tmp/DWSF# python generate_segdata.py --img_path coconew/val --out_path watermarked --
weight_path outputnew/pth
  0%|                                                                                                                    | 0/324 [00:00<?, ?it/s]
[ WARN:0@5.978] global loadsave.cpp:848 imwrite_ Unsupported depth image for selected encoder is fallbacked to CV_8U.
100%|                                                                                                                    | 324/324 [00:18<00:00, 17.45it/s]
(base) root@autodl-container-a3c347aab8-8e755a96:~/autodl-tmp/DWSF#
```

2. 编写代码split.py，将生成的带水印的图像和掩码按8:2分为训练集和训练集

3. 使用生成的数据训练分割模型

```
(base) root@autodl-container-a3c347aab8-8e755a96:~/autodl-tmp/DWSF# python train_seg.py --train_path train_watermarked --test_path test_watermarked --output_path pth_output
/root/miniconda3/lib/python3.8/site-packages/torch/nn/_reduction.py:42: UserWarning: size_average and reduce args will be deprecated, please use reduction='none' instead.
  warnings.warn(warning.format(ret))
/root/miniconda3/lib/python3.8/site-packages/torch/nn/functional.py:3631: UserWarning: Default upsampling behavior when mode=bilinear is changed to align_corners=False since 0.4.0. Please specify align_corners=True if the old behavior is desired. See the documentation of nn.Upsample for details.
  warnings.warn(
[Train]Epoch: 0, BCE loss: 417.03024542331696,
[Train]Epoch: 0, IOU loss: 53.80021245777607,
[Val]BCE loss: 98.3201043009758,
[Val]IOU loss: 13.203404977917671,
[Train]Epoch: 1, BCE loss: 394.50334644317627,
[Train]Epoch: 1, IOU loss: 52.641159147024155,
[Val]BCE loss: 97.1864102780819,
[Val]IOU loss: 13.061682999134064,
[Train]Epoch: 2, BCE loss: 368.9330277442932,
[Train]Epoch: 2, IOU loss: 52.33612921833992,
```

```
[Val]BCE loss: 163.25832206755877,
[Val]IOU loss: 9.890535835176706,
[Train]Epoch: 94, BCE loss: 157.85936572030187,
[Train]Epoch: 94, IOU loss: 26.60589058510959,
[Val]BCE loss: 127.82102503627539,
[Val]IOU loss: 9.942372862249613,
[Train]Epoch: 95, BCE loss: 159.12478490546346,
[Train]Epoch: 95, IOU loss: 25.839709687046707,
[Val]BCE loss: 108.12829201668501,
[Val]IOU loss: 8.788833897560835,
[Train]Epoch: 96, BCE loss: 159.58399285376072,
[Train]Epoch: 96, IOU loss: 26.35874004662037,
[Val]BCE loss: 109.41150930523872,
[Val]IOU loss: 9.802934970706701,
[Train]Epoch: 97, BCE loss: 140.0451769977808,
[Train]Epoch: 97, IOU loss: 23.704677186906338,
[Val]BCE loss: 177.91791360266507,
[Val]IOU loss: 10.706087276339531,
[Train]Epoch: 98, BCE loss: 143.98412219807506,
[Train]Epoch: 98, IOU loss: 24.8625029893592,
[Val]BCE loss: 170.8515056744218,
[Val]IOU loss: 10.535456301644444,
[Train]Epoch: 99, BCE loss: 130.8351082764566,
[Train]Epoch: 99, IOU loss: 22.198526212014258,
[Val]BCE loss: 135.62680569291115,
[Val]IOU loss: 10.689042706042528,
```

结果分析

- 训练集 BCE 损失

训练集的 BCE 损失整体呈现下降趋势。从初始的 417.03 左右逐渐降低到 130.84 左右，这表明随着训练轮数的增加，模型在训练数据上对于每个像素分类的准确性在不断提高，模型正在学习到数据的特征并不断优化。

- 训练集 IOU 损失

训练集的 IOU 损失同样整体呈下降趋势，从最初的 53.80 左右下降到 22.20 左右。这说明模型预测的分割区域与真实分割区域的重叠程度在不断改善，模型的分割能力在逐渐增强。

- 验证集 BCE 损失

验证集的 BCE 损失没有呈现出明显的单调下降趋势，而是在一个范围内波动。

- 验证集 IOU 损失

验证集的 IOU 损失也有波动，不过整体上波动幅度相对较小。

3. 评估模型

```
(base) root@autodl-container-a3c347aab8-8e755a96:~/autodl-tmp/DWSF# python evaluate.py --ori_path coconew/val --pth_path outputnew/pth --out_pa
th output_end
Evaluate Time: Thu Jun 12 13:25:37 2025
===============Encoding===============
100%|[redacted]| 324/324 [01:03<00:00,  5.13it/s]
===============Decoding===============
===============Test: ['Identity()'] ==========
  0%|[redacted]| 0/324 [00:00<?, ?it/s]
/root/miniconda3/lib/python3.8/site-packages/torch/nn/functional.py:3631: UserWarning: Default upsampling behavior when mode=bicubic is changed
 to align_corners=False since 0.4.0. Please specify align_corners=True if the old behavior is desired. See the documentation of nn.Upsample for
 details.
  warnings.warn(
/root/miniconda3/lib/python3.8/site-packages/torch/nn/functional.py:3631: UserWarning: Default upsampling behavior when mode=bilinear is change
d to align_corners=False since 0.4.0. Please specify align_corners=True if the old behavior is desired. See the documentation of nn.Upsample fo
r details.
  warnings.warn(
/root/miniconda3/lib/python3.8/site-packages/torch/nn/functional.py:1806: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid i
nstead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
100%|[redacted]| 324/324 [00:51<00:00,  6.35it/s]
=======result:['Identity()']=======
psnr: 41.77532449180697  ssim: 0.9759778006054047        len:324
MIN: bit_acc: 0.5926954732510287          bit_check_acc: 0.07098765432098765
MEAN: bit_acc: 0.5036643811720971         bit_check_acc: 0.07098765432098765
FUSION: bit_acc: 0.5030864197530864       bit_check_acc: 0.07407407407407407
```

```
===============Test: ['Joint([RandomCrop(0.7,1), RandomResize(0.5,2)])'] ==========
100%|[redacted]| 324/324 [00:47<00:00,  6.76it/s]
=======result:['Joint([RandomCrop(0.7,1), RandomResize(0.5,2)])']=======
psnr: 41.77532449180697  ssim: 0.9759778006054047        len:324
MIN: bit_acc: 0.5104938271604939          bit_check_acc: 0.021604938271604937
MEAN: bit_acc: 0.49421296296296297        bit_check_acc: 0.021604938271604937
FUSION: bit_acc: 0.49516460905349796      bit_check_acc: 0.024691358024691357
===============Test: ['Joint([RandomOcclusion(0.25,0.5), RandomJpegTest(50,100)])'] ==========
100%|[redacted]| 324/324 [00:53<00:00,  6.05it/s]
=======result:['Joint([RandomOcclusion(0.25,0.5), RandomJpegTest(50,100)])']=======
psnr: 41.77532449180697  ssim: 0.9759778006054047        len:324
MIN: bit_acc: 0.49197530864197525         bit_check_acc: 0.0
MEAN: bit_acc: 0.49197530864197525        bit_check_acc: 0.0
FUSION: bit_acc: 0.49197530864197525      bit_check_acc: 0.0
===============Test: ['Joint([RandomCrop(0.7,1), RandomResize(0.5,2), RandomJpegTest(50,100)])'] ==========
100%|[redacted]| 324/324 [00:54<00:00,  5.93it/s]
=======result:['Joint([RandomCrop(0.7,1), RandomResize(0.5,2), RandomJpegTest(50,100)])']=======
psnr: 41.77532449180697  ssim: 0.9759778006054047        len:324
MIN: bit_acc: 0.5004115226337449          bit_check_acc: 0.006172839506172839
MEAN: bit_acc: 0.5004115226337449         bit_check_acc: 0.006172839506172839
FUSION: bit_acc: 0.5004115226337449       bit_check_acc: 0.006172839506172839
===============Test: ['Joint([RandomCrop(0.7,1),RandomOcclusion(0.25,0.5),RandomJpegTest(50,100)])'] ==========
100%|[redacted]| 324/324 [00:49<00:00,  6.53it/s]
=======result:['Joint([RandomCrop(0.7,1),RandomOcclusion(0.25,0.5),RandomJpegTest(50,100)])']=======
psnr: 41.77532449180697  ssim: 0.9759778006054047        len:324
MIN: bit_acc: 0.49742798353909473         bit_check_acc: 0.006172839506172839
MEAN: bit_acc: 0.49742798353909473        bit_check_acc: 0.006172839506172839
FUSION: bit_acc: 0.49742798353909473      bit_check_acc: 0.006172839506172839
```

> **评估指标说明**
>
> 1. PSNR（峰值信噪比）：衡量图像失真程度，值越高表示图像质量保留越好（本次结果均为 41.78，说明原始图像质量保持良好）。
> 2. SSIM（结构相似性）：衡量图像结构相似度，值越接近 1 表示与原图越相似（本次结果均为 0.976，表明图像结构保留优秀）。
> 3. bit_acc（比特准确率）：提取的水印比特与原始比特的一致性，值越高表示水印鲁棒性越强。

> 4. bit_check_acc（比特检查准确率）：可能是对特定水印校验规则的准确率，本次结果普遍较低。

评估结果分析

一、单一攻击测试分析

1. 无攻击（Identity ()）

- bit_acc：MIN=0.593，MEAN=0.504，FUSION=0.503

  无攻击时，水印提取准确率约 50%，接近随机猜测（50%），可能存在水印嵌入强度不足或提取算法问题。

2. JPEG 压缩（RandomJpegTest (50,100)）

- bit_acc：约 0.492

  JPEG 压缩对水印影响显著，准确率略低于无攻击场景，表明水印对有损压缩敏感。

3. 高斯噪声（RandomGN (3,10)）

- bit_acc：MIN=0.592，MEAN=0.499

  低强度高斯噪声下，水印提取准确率波动较大，部分情况接近无攻击表现。

4. 高斯模糊（RandomGF (3,8)）

- bit_acc：约 0.492

  模糊操作对水印有明显破坏，准确率与 JPEG 压缩相当。

5. 颜色调整（RandomColor (0.5,1.5)）

- bit_acc：MIN=0.579，MEAN=0.498

  颜色空间变换对水印影响中等，亮度 / 对比度调整可能改变像素值分布，影响水印提取。

6. 随机裁剪（RandomCrop (0.7,1)）

- bit_acc：MIN=0.586，MEAN=0.503

  裁剪攻击下，水印准确率略有提升，可能因裁剪区域未覆盖水印主要嵌入位置。

7. 随机旋转（RandomRotate (30)）

- bit_acc：MIN=0.591，MEAN=0.504

  旋转操作对水印影响较小，可能因算法设计时考虑了几何变换鲁棒性。

二、联合攻击测试分析

1. 颜色调整 + JPEG 压缩

- bit_acc：约 0.492

  组合攻击未显著降低准确率，可能因两种攻击对水印的破坏机制有重叠。

2. 裁剪 + JPEG 压缩

- bit_acc：约 0.491

  裁剪后再压缩对水印的破坏与单一攻击相当，未产生叠加效应。

3. 裁剪 + Resize+JPEG 压缩

- bit_acc：约 0.500

  几何变换与压缩的组合攻击下，水印准确率反而略有提升，可能因多步处理后水印特征更稳定。

4. 遮挡 + JPEG 压缩

- bit_acc：约 0.491，bit_check_acc=0

  遮挡攻击对水印破坏严重，结合压缩后几乎无法正确提取水印校验位。

三、结论说明

1. 图像质量保留良好：PSNR 和 SSIM 值表明，水印嵌入对原始图像的视觉质量影响极小，**系统在保真度上表现优秀**。
2. **水印鲁棒性不足**：
   - 单一攻击下，比特准确率普遍接近 50%（随机水平），说明水印嵌入强度或提取算法有待优化。
   - 联合攻击未显著降低准确率，可能暗示攻击之间存在抵消效应，或水印分布具有一定抗组合攻击能力。
3. 几何变换鲁棒性优于像素操作：旋转、裁剪等几何攻击下的准确率略高于噪声、压缩等像素级攻击，说明**系统对几何变换的鲁棒性设计更完善**。
4. 校验机制脆弱：bit_check_acc 普遍较低（接近 0），表明水印的校验位在攻击下极易丢失。